



UT-4

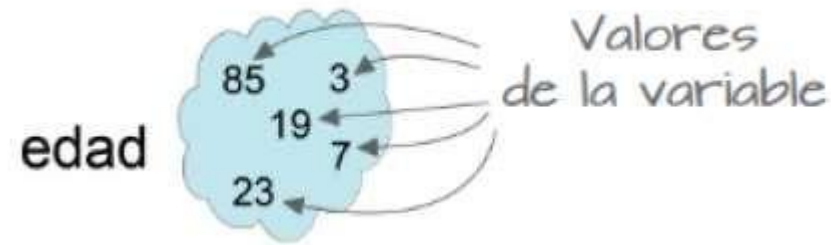
Vectores (Arrays y Matrices)

1º DAM/DAW
(2024-2025)



1 – Variables Escalares Vs. Vectores

Un vector, es un tipo de dato estructurado que permite almacenar un conjunto de datos homogéneo, es decir, todos ellos del mismo tipo y relacionados. Cada uno de los elementos que componen un vector pueden ser de tipo simple como caracteres, entero o real, o de tipo compuesto o estructurado como son vectores, estructuras, listas...



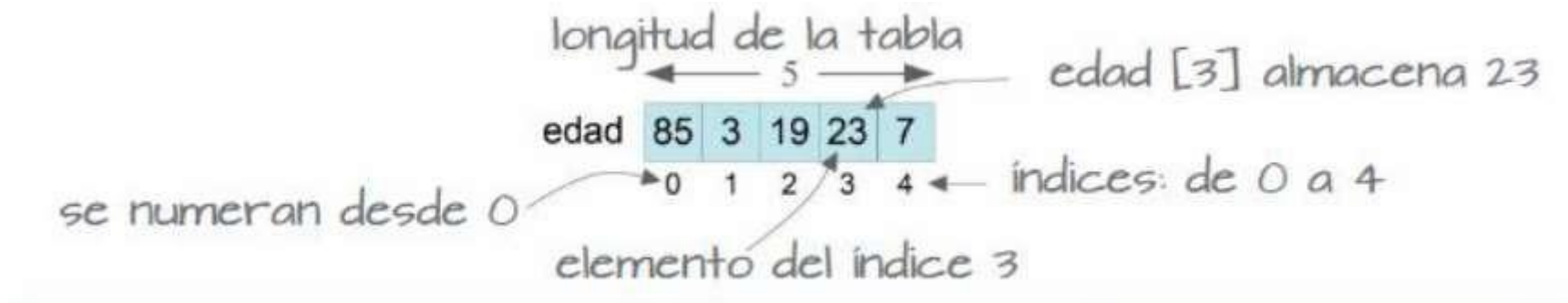
A los datos almacenados en un vector se les denomina elementos; al número de elementos de un vector se les denomina tamaño o rango del vector.

Por norma general emplearemos un vector cuando necesitemos usar un conjunto de datos del mismo tipo que estén relacionados entre sí.

A estas estructuras de datos también se las denomina Tablas, Arrays o Arreglos (este último termino se desaconseja).

2 – Índices de un vector

Distinguimos cada elemento almacenado en un vector por la posición que ocupa en el mismo. A esta posición se le denomina índice. El primer elemento tendrá el índice 0, el segundo elemento el 1 y así sucesivamente.



La forma de utilizar un elemento concreto de un vector es por medio del nombre de la variable que le hemos dado al vector seguido del índice entre corchetes [] que identifica al elemento.

```
System.out.println(edad[0]); // Muestra el primer elemento : 85
edad[4] = 8; // Asigna un nuevo valor al quinto elemento.
```

Si empleamos un índice que no se corresponde al rango del vector generaremos un error en tiempo de ejecución y nuestro programa terminará. Por ejemplo `edad[-1]` o `edad[20]`

3 – Construcción de vectores

En el momento de crear un vector deberemos tener en cuenta lo siguiente.

1. Decidir qué tipo de datos vamos a almacenar
2. Declarar un nombre para el vector

```
tipo nombreVariable[];  
tipo[] nombreVariable;
```

```
int edad [];
```

```
int [] edad;
```

3. Crear la propia tabla.

Con new

```
nombreVariable = new tipo[longitud];
```

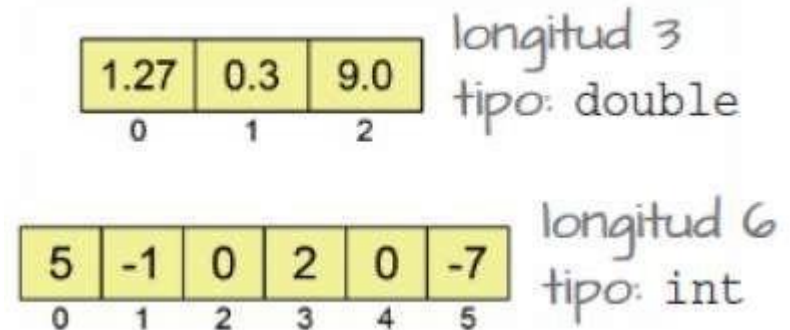
```
edad = new int [5]; // Vector de rango 5.
```

Alternativa

```
int datos [] = {2,-3,0,7}; // Vector de rango 4
```

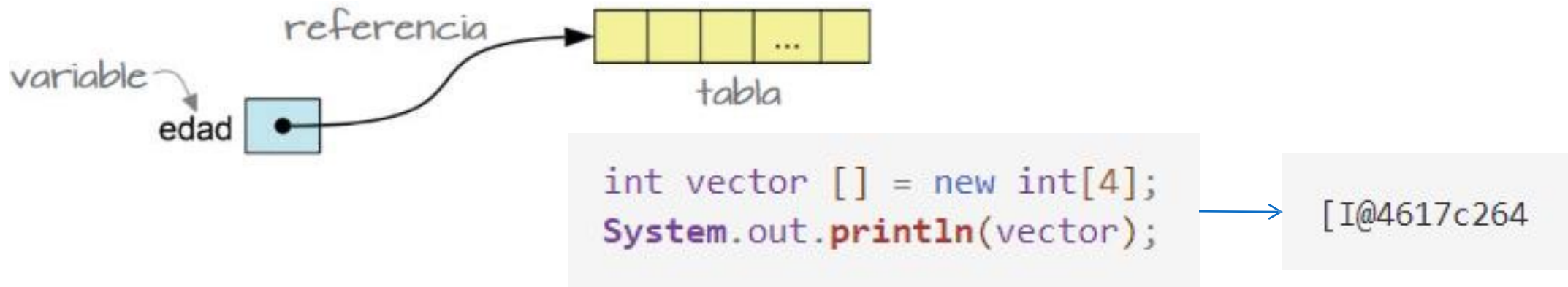


```
int datos []; // Declaramos la variable  
datos = new int [4]; // Creamos el vector  
datos[0] = 2; // Asignamos valores  
datos[1] = -3;  
datos[2] = 0;  
datos[3] = 7;
```

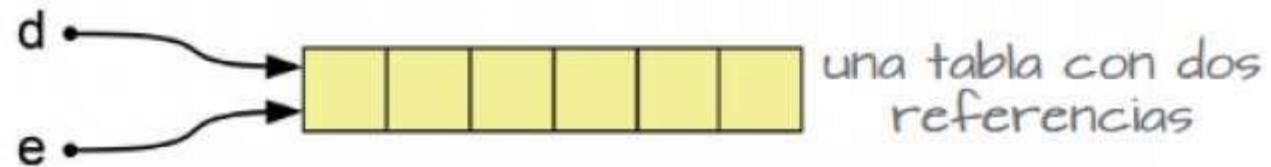


4 – Referencia

En el caso de los vectores, las variables no contienen el valor de todos los elementos del vector sino la dirección de memoria donde están guardadas, es lo que denominamos referencia.



```
int d[], e[]; //variables  
d = new int[6]; //construimos una tabla referenciada por d  
e = d; //ahora la variable e referencia la misma tabla que d. Ambas guardan  
//la misma dirección de memoria
```



5 – Operaciones con vectores (Clase Arrays)

La API de Java nos proporciona herramientas que facilitan el trabajo con vectores. Del mismo modo que usamos la clase Math o Scanner, podemos usar la clase Arrays que contiene funciones para trabajar con vectores.

Obtención del número de elementos de un vector

nombreVariable.length

```
int notas [] = new int[10];  
System.out.println(notas.length);
```



10

Inicialización de valores de un vector (2 métodos)

```
float floats[] = new float[5];
```

```
char[] chars = new char[5];
```

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

```
int[] myNum = {10, 20, 30, 40};
```

5 – Operaciones con vectores (Clase Arrays)

Recorrer un vector

Por lo general, para recorrer todos los elementos de un vector emplearemos un bucle FOR. Existen dos formas de empleo.

```
int alturas [] = {134,156,190,176,167};  
// Para recorrer todos los elementos de este vector podemos usar...  
  
// El for clásico :  
for(int i=0; i<alturas.length;i++){  
    System.out.println(alturas[i]);  
}  
  
// El for-each  
for(int medida: alturas){  
    System.out.println(medida);  
}
```

5 – Operaciones con vectores (Clase Arrays)

Mostrar el contenido de un vector

Podemos mostrar el contenido de un vector recorriéndolo e imprimiendo cada uno de sus elementos (como en el ejemplo anterior), pero la clase Arrays nos proporciona el método `Arrays.toString()` para imprimirlos de una forma más sencilla.

```
int alturas [] = {134,156,190,176,167};  
System.out.println(Arrays.toString(alturas));
```

```
[134, 156, 190, 176, 167]
```


5 – Operaciones con vectores (Clase Arrays)

Copia de un vector

La clase Arrays no proporciona dos métodos para copiar vectores.

- El método `Arrays.copyOf(origen, longitud)`. Construye y devuelve una copia de origen con la longitud especificada.

```
int t[] = {1,2,1,6,23}; // tabla origen
int a[], b[]; // tablas destino
a = Arrays.copyOf(t,newLength:3); // a = [1, 2, 1]
b = Arrays.copyOf(t,newLength:10); // b = [1, 2, 1, 6, 23, 0, 0, 0, 0, 0]
```

- El método `Arrays.copyOfRange(origen, desde, hasta)`. Construye y devuelve un vector donde se ha copiado los elementos de

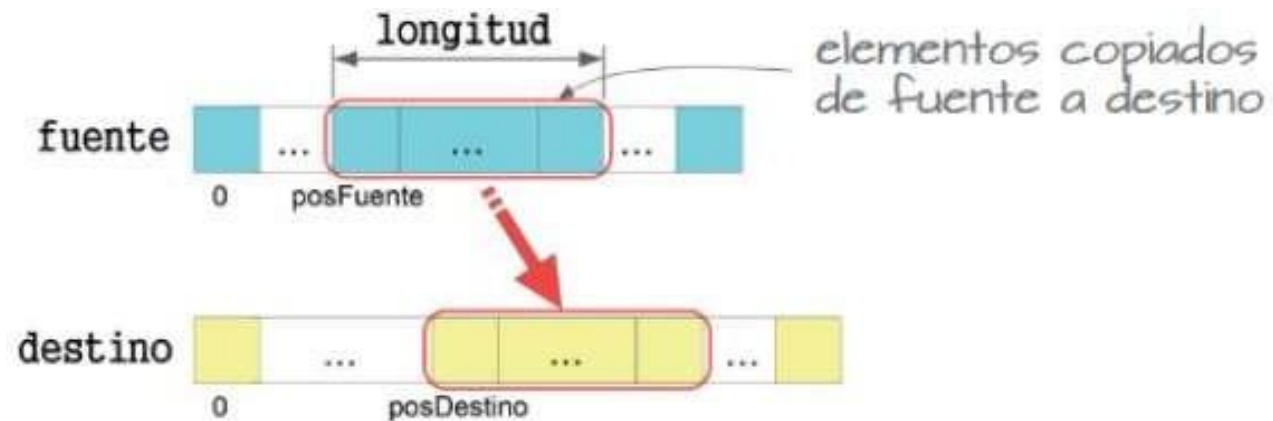
```
int t[] = {7,5,3,1,0,-2}; // tabla origen
int a[];
a = Arrays.copyOfRange(t,from:1,to:4); // a = [5, 3, 1]
```

5 – Operaciones con vectores (Clase Arrays)

Copia de un vector

La clase `System` también proporciona un método para copiar elementos entre vectores `System.arraycopy()`. En este caso, a diferencia de los métodos de la clase `Arrays`, no se crea ningún vector nuevo (ya deben estar creados) y los elementos en el vector destino serán sobrescritos. La sintaxis de este método es la siguiente.

`System.arraycopy(fuente, posFuente, destino, posDestino, longitud);`



5 – Operaciones con vectores (Clase Arrays)

Comparar el contenido de dos vectores

No podemos comparar el contenido de dos vectores mediante el operador `==` ya que de esta manera lo que comparamos es la referencia del vector. Por ejemplo:

```
int t1[] = {7,5,3,1}; //  
int t2[] = {7,5,3,1}; // t1 y t2 tienen los mismos elementos  
System.out.println(t1==t2); // Esta instrucción mostrará FALSE
```

Por ello empleamos el método `Arrays.equals (vector1, vector2)`. Para comparar los vectores anteriores usaremos:

```
System.out.println(Arrays.equals(t1,t2)); // Pero esta instrucción mostrará TRUE
```

Ejemplo - Array con zona útil

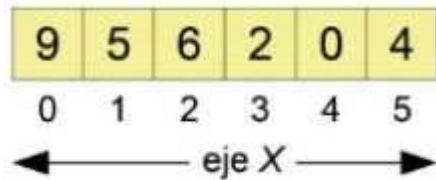
```
int[] array = {2, 4, 8, 12, 16, 18, 20, 10, 9};  
array[5]=0;  
int i = 0;  
while (array[i]!=0 && i<array.length) {  
    System.out.println(array[i]);  
    i++;  
}
```

Array	
0	2
1	4
2	8
3	12
4	16
5	0
6	20
7	10
8	9

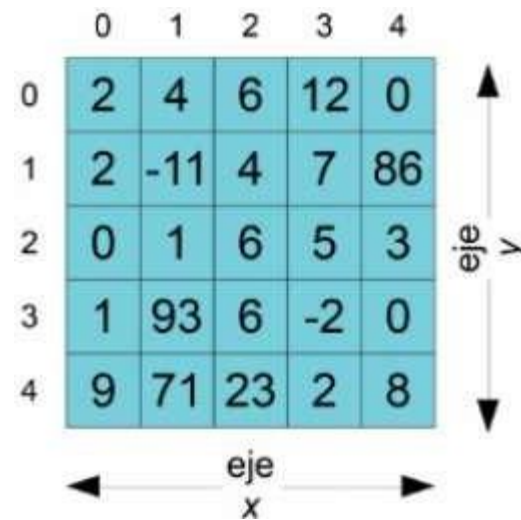
6 – Vectores n-dimensionales (Matrices)

En los ejemplos vistos hasta ahora los vectores solo tenían una dimensión (solo tienen longitud) y basta con un índice para recorrerlos. Pero puede ocurrir que necesitemos vectores con más de una dimensión.

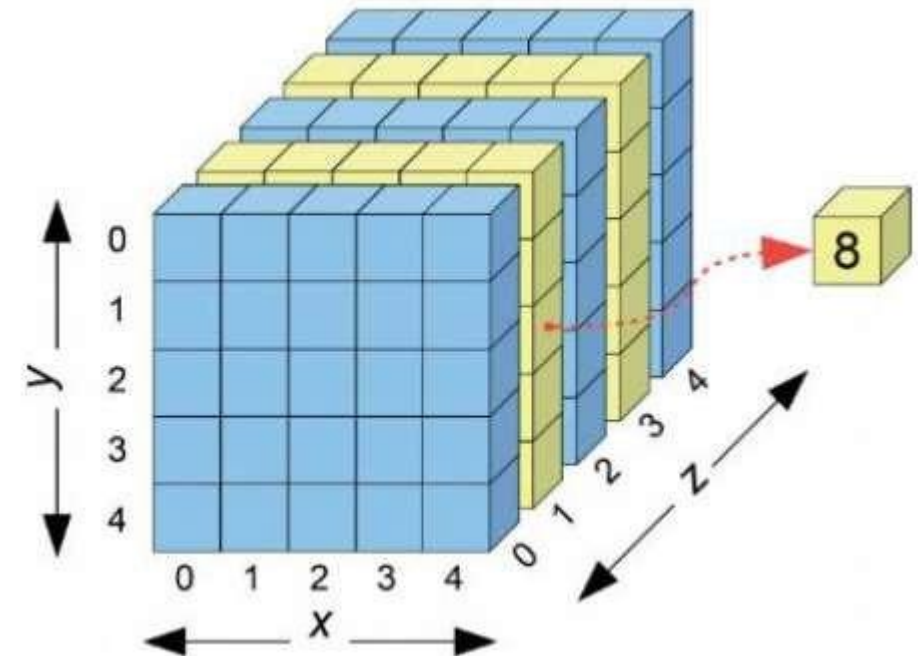
1-dimensión



2-dimensión



3-dimensión



6 – Vectores n-dimensionales (Matrices)

En el caso de los vectores n-dimensionales los declararemos del mismo modo que los de una dimensión pero añadiendo un conjunto de corchetes por cada una de las dimensiones extra.

2-dimensión

	0	1	2	3	4
0	2	4	6	12	0
1	2	-11	4	7	86
2	0	1	6	5	3
3	1	93	6	-2	0
4	9	71	23	2	8

Diagram illustrating a 2D matrix (5x5) with axes labeled 'eje x' (horizontal) and 'eje y' (vertical).

```
// Declaración de matriz 5x5  
int matriz [][];  
matriz = new int [5][5];
```

filas ↑ ↑ columnas

```
// Declaración de matriz 5x5 con valores  
int matriz [][] = {{2,4,6,12,0},{2,-11,4,7,86},{0,1,6,5,3},{1,93,6,-2,0},{9,71,23,2,8}};
```

6 – Vectores n-dimensionales (Matrices)

Para recorrer una matriz bidimensional necesitaremos dos índices, el del eje X y el del eje Y.

2-dimensión

	0	1	2	3	4
0	2	4	6	12	0
1	2	-11	4	7	86
2	0	1	6	5	3
3	1	93	6	-2	0
4	9	71	23	2	8

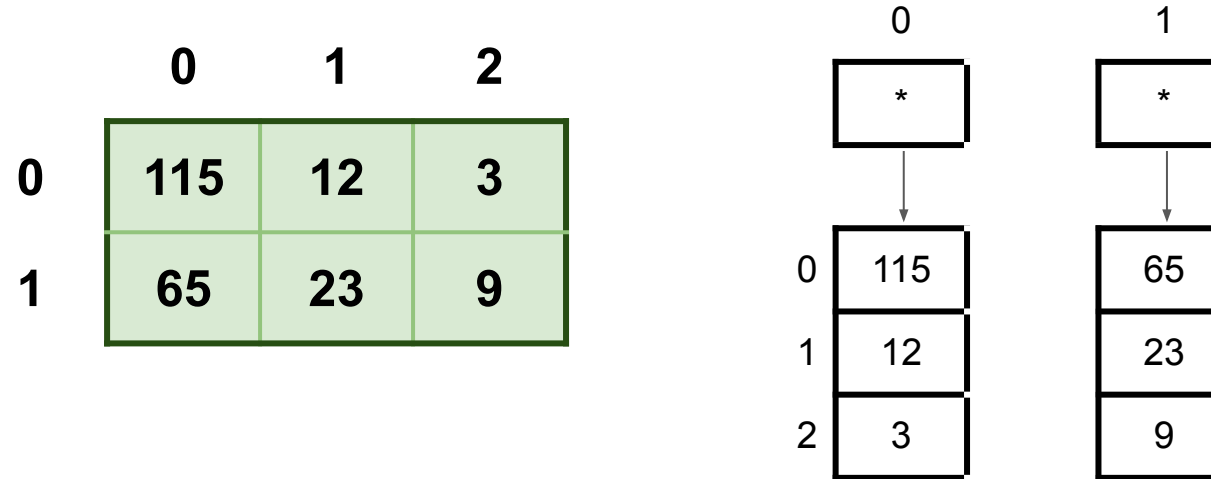
Diagram illustrating a 2D matrix (5x5) with indices 0 to 4 for both axes. The horizontal axis is labeled 'eje x' and the vertical axis is labeled 'eje y'.

```
for (int i=0; i<5; i++){  
    for (int j=0; j<5; j++){  
        System.out.println(matriz[i][j]);  
    }  
}
```

Para mostrar sus elementos podemos hacerlo como en el ejemplo anterior o emplear el método `Arrays.deepToString()`.

```
System.out.println(Arrays.deepToString(matriz));
```

Tamaño de la matriz



La variable que almacena un array es una referencia a una serie de posiciones sucesivas con la información del array.

En una matriz la variable es una referencia a una serie de arrays, es decir a una serie de referencias a los conjuntos de información. Entonces:

`matriz.lenght`: mide el número de arrays que contiene la matriz. Es decir, el número de filas.

`matriz[i].lenght`: mide el número de elementos del array `i` de la matriz. Es decir, el número de columnas.

Recorrer una fila

```
int[][] matriz = {{115, 12, 3}, {65, 23, 9}};  
System.out.println();  
for (int i = 0; i < matriz[0].length; i++) {  
    System.out.println(matriz[0][i]);  
}
```

	0	1	2
0	115	12	3
1	65	23	9

Recorrer una columna

```
int[][] matriz = {{115,12,3},{65,23,9}};  
for (int i = 0; i<matriz.length;i++ ) {  
    System.out.println(matriz[i][0]);  
}
```

	0	1	2
0	115	12	3
1	65	23	9